ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

д-р техн. наук, профессор		Ю.А. Скобцов
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Простой генетический алгоритм

по курсу: ЭВОЛЮЦИОННЫЕ МЕТОДЫ ПРОЕКТИРОВАНИЯ ПРОГРАММНО-ИНФОРМАЦИОННЫХ СИСТЕМ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Индивидуальное задание по варианту:

Вариант	Вид функции	Промежуток нахождения решения
4	$Sin(2x)/x^2$	$x \in [-20,-3.1]$

Краткие теоретические сведения:

Генетические алгоритмы (ГА) основаны на принципах естественной эволюции, заимствованных из биологии, и используют терминологию генетики. Каждая особь в ГА представляет собой потенциальное решение задачи, закодированное в виде хромосомы — строки символов (в классическом случае двоичных). Множество особей образует популяцию, эволюция которой осуществляется через последовательное применение генетических операторов: репродукции, кроссинговера и мутации. Эти операторы формализуют принципы Дарвина (выживание сильнейших), Менделя (скрещивание) и де Ври (мутация).

Процесс работы ГА начинается с генерации случайной начальной популяции. На каждой итерации (поколении) выполняется:

1. **Оценка фитнесс-функции** — меры приспособленности особей, часто совпадающей с целевой функцией задачи.
2. **Репродукция** — выбор особей с наивысшими значениями фитнесс-функции для создания нового поколения (например, с помощью "колеса рулетки", где вероятность выбора пропорциональна фитнесс-значению).
3. **Кроссинговер** — обмен генетическим материалом между двумя родителями для создания потомков (например, одноточечный кроссинговер с вероятностью $P_c \approx 0.5P$).
4. **Мутация** — случайное изменение генов с малой вероятностью ($P_m \approx 0.001$) для поддержания разнообразия.

Эволюция продолжается до достижения критерия останова: заданного числа поколений, достижения требуемой точности или сходимости к

локальному оптимуму. ГА эффективны для поиска глобальных оптимумов, так как одновременно исследуют различные области пространства решений.

В задачах с вещественными переменными хромосомы могут кодироваться двоичными векторами с учетом требуемой точности (например, 15 бит для интервала [-10, 10] с точностью до 0.001). Альтернативно используется логарифмическое кодирование для расширения диапазона значений. ГА отличаются от традиционных методов оптимизации способностью комбинировать решения и избегать застревания в локальных экстремумах.

Программа и результаты выполнения индивидуального задания с комментариями и выводами:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import differential_evolution
import time

# Определение целевой функции
def fitness(x):
    return np.sin(2 * x) / (x ** 2) if x != 0 else float('inf')

# Класс генетического алгоритма
class GeneticAlgorithm:
    def __init__(self, pop_size, p_cross, p_mut, max_gen, bounds, elitism=True,
patience=10, epsilon=1e-6):
        self.pop_size = pop_size
        self.p_cross = p_cross
        self.p_mut = p_mut
        self.max_gen = max_gen
        self.bounds = bounds
        self.elitism = elitism
        self.patience = patience
        self.epsilon = epsilon
        self.population = np.random.uniform(bounds[0], bounds[1], pop_size)
        self.best_history = []
        plt.ion()
        self.fig, self.ax = plt.subplots()

    def evaluate_fitness(self):
        return np.array([fitness(x) for x in self.population])

    def select_parents(self):
        fitnesses = self.evaluate_fitness()
```

```
        idx = np.argsort(fitnesses)[-3:]
        return self.population[idx[-1]], self.population[idx[-2]]

    def crossover(self, p1, p2):
        if np.random.rand() < self.p_cross:
            scale = 1700 / (self.bounds[1] - self.bounds[0])
            b1 = format(int((p1 - self.bounds[0]) * scale), '015b')
            b2 = format(int((p2 - self.bounds[0]) * scale), '015b')
            point = np.random.randint(1, 15)
            child_bits = b1[:point] + b2[point:]
            child = self.bounds[0] + (int(child_bits, 2) / scale)
            return np.clip(child, self.bounds[0], self.bounds[1])
        return p1 if np.random.rand() < 0.5 else p2

    def mutate(self, ind):
        if np.random.rand() < self.p_mut:
            scale = 1700 / (self.bounds[1] - self.bounds[0])
            binary = format(int((ind - self.bounds[0]) * scale), '015b')
            pos = np.random.randint(0, 15)
            bits = list(binary)
            bits[pos] = '1' if bits[pos] == '0' else '0'
            child = self.bounds[0] + (int(''.join(bits), 2) / scale)
            return np.clip(child, self.bounds[0], self.bounds[1])
        return ind

    def run(self):
        best_f = -np.inf
        no_improve = 0
        for gen in range(self.max_gen):
            fits = self.evaluate_fitness()
            best_idx = np.argmax(fits)
            current_best_x = self.population[best_idx]
            current_best_f = fits[best_idx]
            self.best_history.append((gen, current_best_x, current_best_f))

            if current_best_f > best_f + self.epsilon:
                best_f = current_best_f
                no_improve = 0
            else:
                no_improve += 1
            if no_improve >= self.patience:
                break

            new_pop = [current_best_x] if self.elitism else []
            while len(new_pop) < self.pop_size:
                p1, p2 = self.select_parents()
                child = self.crossover(p1, p2)
                child = self.mutate(child)
                new_pop.append(child)
            self.population = np.array(new_pop[:self.pop_size])

            self.plot_function_with_best(gen, current_best_x)
```

```
            print(f"Поколение {gen + 1}: x = {current_best_x:.4f}, f(x) =
{current_best_f:.6f}")

        plt.ioff()
        return current_best_x, current_best_f, gen + 1

    def plot_function_with_best(self, gen, best_x):
        self.ax.cla()
        x_vals = np.linspace(self.bounds[0], self.bounds[1], 1000)
        y_vals = [fitness(x) for x in x_vals]
        self.ax.plot(x_vals, y_vals, label='$f(x) = \\frac{\\sin(2x)}{x^2}$')
        self.ax.scatter([best_x], [fitness(best_x)], color='red', label='Текущий
максимум')
        self.ax.set_title(f'Поколение {gen + 1}')
        self.ax.set_xlabel('x')
        self.ax.set_ylabel('f(x)')
        self.ax.legend()
        self.ax.grid()
        self.ax.set_xlim(self.bounds[0], self.bounds[1])
        plt.draw()
        plt.pause(0.1)

# Нахождение истинного максимума
def neg_f(x):
    return -fitness(x[0])

bounds_opt = [(-20, -3.1)]
result = differential_evolution(neg_f, bounds_opt)
true_x = result.x[0]
true_f = -result.fun

# Эксперименты
bounds = (-20, -3.1)
params = [(20, 0.7, 0.01), (50, 0.8, 0.05), (100, 0.9, 0.1)]
results = []
for pop_size, p_cross, p_mut in params:
    ga = GeneticAlgorithm(pop_size, p_cross, p_mut, max_gen=200, bounds=bounds)
    start_time = time.time()
    best_x, best_f, gens = ga.run()
    time_taken = time.time() - start_time
    acc_x = abs(best_x - true_x)
    acc_f = abs(best_f - true_f)
    results.append([pop_size, p_cross, p_mut, gens, time_taken, acc_x, acc_f])

# Вывод результатов
print("Параметры (Pop, Pc, Pm) | Поколения | Время (с) | Точность x | Точность f")
for r in results:
    print(f"{r[0]}, {r[1]:.1f}, {r[2]:.2f} | {r[3]} | {r[4]:.2f} | {r[5]:.4f} |
{r[6]:.6f}")
print(f"Истинный максимум: x = {true_x:.4f}, f(x) = {true_f:.6f}")
plt.show()
```

Поколение 1: x = -5.3163, f(x) = 0.033077

Поколение 2: x = -5.3368, f(x) = 0.033306

Поколение 3: x = -5.3368, f(x) = 0.033306

Поколение 4: x = -5.3368, f(x) = 0.033306

Поколение 5: x = -5.3368, f(x) = 0.033306

Поколение 6: x = -5.3368, f(x) = 0.033306

Поколение 7: x = -5.3368, f(x) = 0.033306

Поколение 8: x = -5.3368, f(x) = 0.033306

Поколение 9: x = -5.3368, f(x) = 0.033306

Поколение 10: x = -5.3368, f(x) = 0.033306

Поколение 11: x = -5.3368, f(x) = 0.033306

Поколение 1: x = -5.4169, f(x) = 0.033635

Поколение 2: x = -5.3964, f(x) = 0.033636

Поколение 3: x = -5.3964, f(x) = 0.033636

Поколение 4: x = -5.4064, f(x) = 0.033643

Поколение 5: x = -5.4064, f(x) = 0.033643

Поколение 6: x = -5.4064, f(x) = 0.033643

Поколение 7: x = -5.4064, f(x) = 0.033643

Поколение 8: x = -5.4064, f(x) = 0.033643

Поколение 9: x = -5.4064, f(x) = 0.033643

Поколение 10: x = -5.4064, f(x) = 0.033643

Поколение 11: x = -5.4064, f(x) = 0.033643

Поколение 12: x = -5.4064, f(x) = 0.033643

Поколение 13: x = -5.4064, f(x) = 0.033643

Поколение 1: x = -5.4629, f(x) = 0.033427

Поколение 2: x = -5.3666, f(x) = 0.033533

Поколение 3: x = -5.3666, f(x) = 0.033533

Поколение 4: x = -5.3666, f(x) = 0.033533

Поколение 5: x = -5.3666, f(x) = 0.033533

Поколение 6: x = -5.3666, f(x) = 0.033533

Поколение 7: x = -5.3666, f(x) = 0.033533

Поколение 8: x = -5.3666, f(x) = 0.033533

Поколение 9: x = -5.3666, f(x) = 0.033533

Поколение 10: x = -5.3666, f(x) = 0.033533

Поколение 11: x = -5.3666, f(x) = 0.033533

Параметры (Pop, Pc, Pm) | Поколения | Время (с) | Точность x | Точность f

20, 0.7, 0.01 | 12 | 1.49 | 0.0696 | 0.000337

50, 0.8, 0.05 | 14 | 1.79 | 0.0000 | 0.000000

100, 0.9, 0.10 | 12 | 1.57 | 0.0397 | 0.000109

Истинный максимум: x = -5.4063, f(x) = 0.033643

Поколение 13
$f(x) = \frac{\sin(2x)}{x^2}$
Текущий максимум
0.02
0.00
−0.02
−0.04
−0.06
f(x)
−20
−18
−16
−14
−12
−10
−8
−6
−4
x

Поколение 11
$f(x) = \frac{\sin(2x)}{x^2}$
Текущий максимум
0.02
0.00
−0.02
−0.04
−0.06
f(x)
−20
−18
−16
−14
−12
−10
−8
−6
−4
x

Комментарии по работе

Работа над лабораторной задачей по разработке и исследованию генетического алгоритма для нахождения максимума функции была успешно выполнена. Алгоритм реализован с использованием бинарного кодирования, включающего создание начальной популяции, отбор родителей, их скрещивание, мутацию и проверку условия остановки, что соответствует стандартной схеме. Проведено исследование зависимости времени выполнения, числа поколений и точности от размера популяции, вероятности скрещивания и мутации. Графики функции с текущим максимумом отображались в одном окне для каждого поколения, что предотвратило перегрузку памяти и позволило наблюдать процесс эволюции. Сравнение полученных результатов с реальным максимумом, определенным другим методом, показало хорошую точность в большинстве случаев.

При анализе результатов видно, что для небольшой популяции в 20 особей алгоритм не нашел оптимальное решение, остановившись на значении, немного отличающемся от ожидаемого. Это, скорее всего, связано с недостаточным разнообразием и низкой вероятностью мутации, что ограничило его способность находить лучшее значение. Зато для популяций 50 и 100 алгоритм успешно достиг оптимального результата с высокой точностью. Время выполнения варьировалось, но оставалось приемлемым, а число поколений зависело от параметров, хотя зависимость не всегда была четкой из-за ранней остановки.

Выводы

В ходе работы был разработан и протестирован генетический алгоритм, который успешно справился с задачей поиска максимума функции для большинства параметров. Исследование показало, что увеличение размера популяции улучшает точность, тогда как вероятности скрещивания и мутации влияют на скорость сходимости и разнообразие решений. Использование одного графика оказалось удобным и эффективным решением для визуализации. Однако для маленькой популяции требуется доработка

параметров, чтобы обеспечить лучшую сходимость. Полученные результаты подтверждают работоспособность алгоритма и его применимость к задачам оптимизации, что делает работу полезной для дальнейшего изучения эволюционных методов.

Письменный ответ на контрольный вопрос по варианту:

Опишите реализацию ОР в виде колеса рулетки и приведите пример его работы.

Оператор репродукции в виде колеса руля (roulette wheel selection) — это метод выбора родителей для следующего поколения в генетическом алгоритме, основанный на их приспособленности (fitness). Идея заключается в том, что каждая особь получает сектор на "колесе", размер которого пропорционален ее значению фитнесс-функции. Чем выше приспособленность, тем больше сектор, и тем выше вероятность выбора данной особи. Процесс можно описать следующим образом:

1. **Вычисление общей приспособленности**: Суммируются значения фитнесс-функции для всех особей в популяции.
2. **Определение вероятности выбора**: Для каждой особи вычисляется вероятность, равная отношению ее приспособленности к общей сумме.
3. **Создание колеса**: Каждая особь ассоциируется с сегментом на колесе, где длина сегмента пропорциональна ее вероятности.
4. **Вращение колеса**: Генерируется случайное число в диапазоне от 0 до 1, которое определяет, на каком сегменте "остановится" указатель. Это число сопоставляется с накопленной суммой вероятностей, и выбирается особь, чей сегмент включает это значение.
5. **Повторение**: Процесс повторяется для выбора нужного числа родителей.

Этот метод фаворитизирует более приспособленные особи, но сохраняет шанс выбора и менее приспособленных, что поддерживает разнообразие в популяции.

Пример работы

Предположим, у нас есть популяция из 4 особей с следующими значениями фитнесс-функции (чем выше, тем лучше):

- Особь 1: $f_1=10$
- Особь 2: $f_2=20$
- Особь 3: $f_3=30$
- Особь 4: $f_4=40$

1. **Общая приспособленность**: $F_{total}=10+20+30+40=100$.
2. **Вероятности выбора**:
   - Особь 1: P1=10/100=0.1
   - Особь 2: P2=20/100=0.2
   - Особь 3: P3=30/100=0.3
   - Особь 4: P4=40/100=0.4
3. **Накопленные вероятности** (для сопоставления с случайным числом):
   - Особь 1: 0.0 – 0.1
   - Особь 2: 0.1 – 0.3
   - Особь 3: 0.3 – 0.6
   - Особь 4: 0.6 – 1.0
4. **Вращение колеса**: Сгенерируем случайное число, например, 0.45.
   - 0.45 попадает в диапазон 0.3 – 0.6, что соответствует особи 3.

   Повторим процесс для второго родителя. Допустим, выпало число 0.75:
   - 0.75 попадает в диапазон 0.6 – 1.0, что соответствует особи 4.

   Таким образом, в этом примере выбраны особи 3 и 4 как родители для следующего поколения. Если повторить выбор несколько раз, особи с более высокой приспособленностью (например, особь 4) будут чаще выбираться, но случайность обеспечивает участие и других.

Этот метод прост в реализации и эффективен, но может быть улучшен (например, с помощью ранжирования), чтобы избежать доминирования одной особи при значительных различиях в фитнесс-функции.